

Neocloud AI Capex Tracker

System Design Memo — Pre-Implementation Review

Author: Kai | Status: Draft for review | Version: 0.5 | Date: 7 April 2026

1. Purpose & Scope

This memo records the system design discussion for a collaborative GitHub project that automatically tracks, extracts, and evaluates AI-related capital expenditure (capex) disclosures from major hyperscalers and neocloud providers. The project will run on a recurring schedule, ingest quarterly and annual filings as they are released, route work across multiple LLM providers under a strict interchange protocol, and present validated results in a shared Excel workbook stored in version control.

The memo is intentionally written before any implementation begins. Its purpose is to lock in architectural decisions, record rejected alternatives with rationale, and surface the open questions that must be answered before coding starts.

2. Goals & Non-Goals

Goals

- Track AI-attributable capex spending across major hyperscalers and pure-play neoclouds on a quarterly cadence.
- Quantify both **volume** (raw spend) and **efficiency** (capex normalized to revenue, capacity, or output) of that spending.
- Operate continuously and autonomously, triggered by the publication of new filings rather than human prompting.
- Minimize ongoing token cost by routing tasks to the cheapest model capable of producing valid output, under a universal interchange protocol that guarantees output integrity regardless of which model produced it.
- Provide a single human-readable artifact (an Excel workbook) that collaborators can open, audit, and extend.
- Maintain a fully auditable trail: every extracted figure must be traceable to a specific quote in a specific source document.
- Support friendly collaboration via GitHub: code, workbook, source archive, and history all in one repository.

Non-Goals (v1)

- Real-time intra-day price or news tracking. Cadence is quarterly.
- Investment recommendations or financial advice of any kind.
- Coverage of private companies that do not publish quarterly disclosures.
- A web dashboard or hosted UI. Presentation is the workbook itself plus optional generated artifacts.

- Multi-user concurrent editing of the workbook. v1 is single-writer.

3. Guiding Principles

- **Provenance over plausibility.** Every figure must carry a verbatim source quote and locator. Numbers without provenance are not released.
- **Mechanical checks before model checks.** Rules and arithmetic catch errors cheaply and deterministically. LLM-based audit is a last-resort layer, not the primary defense.
- **Single source of truth for validation logic.** Validation rules live in the Excel template as formulas, where humans can read and edit them.
- **Model-agnostic interchange.** Every layer that touches an LLM speaks one versioned protocol. Swapping models is a config change, not a refactor.
- **Cheap watcher, expensive worker.** The always-on scheduling layer costs nothing; LLM inference runs only when a new filing is detected.
- **Defer features that aren't justified yet.** Multi-model routing, eval agents, and computer-use scraping are designed for but not built on day one.

4. Architecture Overview

The system is decomposed into nine layers, each independently testable and replaceable. Data flows top-to-bottom; the workbook serves simultaneously as the storage layer, the validation engine, and the presentation surface.

Layer	Responsibility
Watcher	Scheduled cron job (GitHub Actions) polling SEC EDGAR and non-US IR pages. Detects new filings a
Ingestion	Downloads filings, normalizes to canonical text, assigns stable page/section IDs, computes source-do
Extraction	Single model-agnostic extract() interface. Pluggable model backends behind one abstraction. Emits st
Workbook (template + live)	Hand-designed Excel template defines structure, formulas, named ranges, and validation rules. A live
Workbook write adapter	openpyxl-based Python module. Writes only into designated input cells of source_data and append-on
Formula evaluation pass	LibreOffice headless recalc between agent write and check-read so Excel remains the single source o
Validation pipeline	Schema validator, provenance verifier (Python text match), consistency rules (Excel formulas), eval ag
Storage / distribution	GitHub repo containing code, template, live workbook, CSV mirror for diff-friendly history, and Git LFS
Optional human interface	Claude for Excel sidebar, opened ad hoc by collaborators against the live workbook for analyst-style q

5. Multi-Model Interchange Protocol

The cost-saving rationale for routing different subtasks to different LLM providers is preserved by enforcing a strict, versioned, model-agnostic data contract. Models are interchangeable if and only if their outputs satisfy the contract mechanically. The protocol has three concerns, each tested separately:

5.1 Syntactic validity

Every model output must parse against a versioned JSON schema. Enforced via Pydantic. Invalid outputs are rejected and the parse error is fed back into a single retry. This is a hard gate — nothing proceeds downstream until syntax passes.

5.2 Provenance / proof-of-work

Every extracted field carries:

- **value** — the numeric or categorical fact
- **quote** — verbatim source span, ≤ 30 words
- **locator** — canonical page ID assigned by the ingestion layer (not by the model)
- **source_doc_hash** — SHA-256 of the source document
- **extraction_type** — one of {direct, inferred, derived}
- **confidence** — model-reported, treated as advisory only
- **protocol_version** — pinned per output for forward compatibility
- **extracting_model** — backend identifier and version

5.3 Semantic validity

Combination of mechanical and model-based checks, layered cheapest-first:

- **Layer A — Schema tests:** unit tests over known-good and known-bad blobs. Milliseconds, no LLM calls.
- **Layer B — Golden-set regression:** hand-labeled fixtures replayed on every prompt or model change.
- **Layer C — Provenance verification:** Python substring match confirming every quoted span exists verbatim in the source PDF at the claimed locator.
- **Layer D — Consistency rules:** declarative range, ratio, and relational checks (e.g. $ai_capex \leq total_capex$; $|QoQ\ delta| < 5x$). Implemented as Excel formulas.
- **Layer E — Eval agent:** a different model family asked narrow yes/no questions: “does the quote at locator X support the claim Y?”
- **Layer F — Cross-source triangulation:** when the same metric appears in 10-Q, press release, and earnings call, formulas check they agree within tolerance.

5.4 Known failure modes & mitigations

- **Fabricated quotes** — mitigated by mandatory substring verification at Layer C.
- **Locator drift across models** — mitigated by injecting canonical `<page id="...">` markers in the prompt; the model echoes the marker rather than counting on its own.
- **Eval agent shares biases with extractor** — mitigated by enforcing different model families and narrow audit prompts.
- **Provenance overhead exceeds savings** — mitigated by measuring tokens-per-extraction per model; routing is only enabled where cost benefit exceeds overhead.

- **Silent regressions** — mitigated by golden-set CI gating any prompt or model change.

6. Workbook Design

The workbook is designed twice: a hand-built **template** defines structure and rules; a derived **live workbook** is populated by the agent on every run. The template author and the workbook populator are different roles with disjoint write surfaces — neither can corrupt the other.

6.1 Sheet structure

Sheet	Role
source_data	Agent write target. One row per (company, period, metric) with provenance columns. The only sheet the ext
checks	Formula-driven consistency checks per row. One column per rule plus an aggregate all_checks_pass boole
derived_metrics	Formula-computed analytics (efficiency ratios, QoQ/YoY deltas, capex/revenue, capex per MW, etc.). Never
golden	Hand-labeled 3-year baseline per company. Same schema as source_data. Used for regression testing.
audit_log	Append-only record of every agent write: who/what/when/which model/which protocol version/which validati
schema	Self-describing protocol contract: allowed fields, types, units, and human-readable rule definitions.
dashboard	Read-only presentation layer pulling from derived_metrics. The view collaborators actually consume.

6.2 Write discipline

- openpyxl writes only into designated input cells. Formula sheets are untouched.
- Excel Tables / structured references (Table[@col]) are used in place of cell references so writes survive row insertions.
- LibreOffice headless is invoked between write and read passes to force formula recalculation.
- The CSV mirror in data/csv/ is regenerated on every run so git diffs remain meaningful despite the binary .xlsx format.
- Sheet protection is enabled on every formula sheet to make accidental human edits impossible without explicit unprotect.

7. Repository & Infrastructure

7.1 Repository layout

```
repo/
  .github/workflows/ # scheduled jobs, CI
  src/
    watcher/ ingestion/ extraction/ validation/ adapters/
  workbook/
    capex_tracker_template.xlsx # canonical structure, hand-built
    capex_tracker.xlsx # live populated workbook
  data/
    csv/ # diff-friendly mirror, auto-regenerated
    sources/ # raw filings (Git LFS)
  tests/
```

```
schema/  golden/
docs/
  design.md  # this memo, kept current
README.md
```

7.2 Why GitHub for both code and workbook

- Single source of truth: collaborators see code, data, workbook, and history in one place.
- Every quarterly run is a commit, giving free git-tracked history of the dataset.
- Concurrency groups in GitHub Actions trivially enforce single-writer serialization on the workbook.
- GitHub Actions free tier covers quarterly cadence with no infra cost.

7.3 Caveats and how they are handled

- **Binary diffs are useless.** Mitigated by committing a CSV mirror alongside every workbook write.
- **Repo bloat.** At quarterly cadence, growth is modest. Source PDFs go into Git LFS; .xlsx stays in main repo until size warrants reconsidering.
- **Merge conflicts.** Workbook is a single-writer artifact. Concurrency groups + a rule that humans never edit on a branch with pipeline commits.
- **Secrets.** All API keys live in GitHub Actions secrets. Nothing committed.

8. Considered & Rejected Alternatives

8.1 Claude for Excel as the automated write interface

Claude for Excel is an interactive Office add-in running in a sidebar inside a logged-in Excel session. Anthropic's documentation confirms there is no REST API, no headless mode, and no server-side invocation path. It cannot be triggered from a cron job. Reclassified as an optional human interface, not a pipeline component.

8.2 Computer-use / screen-control automation of Claude for Excel

Technically feasible via Anthropic's Computer Use API or libraries like pyautogui, Playwright, UiPath, Self-Operating Computer, etc. Rejected for v1 because:

- Token cost is significantly higher than openpyxl (vision-token loop on every action).
- UI automation is brittle: any pixel-level layout shift, dialog popup, or render delay breaks the run silently.
- Always-on Windows VM with M365 license adds infra cost and patch maintenance burden.
- Credentials sitting in a desktop session reachable by an autonomous agent is a meaningful security regression versus an environment-variable API key.
- The functional benefit over openpyxl is zero — the pipeline writes structured rows; conversational intelligence is wasted on that task.

Note: computer-use agents may legitimately enter the project later as a watcher-layer tool for scraping non-US IR websites that have no clean API. That is a separate, narrower use case to revisit.

8.3 Multi-model routing on day one

Deferred. The protocol is designed so multi-model routing is a configuration change, not a refactor, but the routing logic itself will not be implemented until measured evidence shows that one model is failing on a specific task or that another model offers cost savings exceeding the orchestration overhead.

9. Open Items

Item	Notes / Status
Golden dataset	Hand-labeled 3-year baseline per company. Scope, methodology, and labeling protocol to be defined.
Excel template design vs openpyxl Detailed design	Detailed cell-level layout, named ranges, input vs formula cell partition, validation rules, and the extraction protocol.
Target company list (v1)	Recommend starting with 3-4 US hyperscalers where SEC EDGAR provides clean source data.
source_data column schema	The protocol contract. Exact field list and types. Highest-leverage open item.
Definition of efficiency metrics	Precise formulas for capex/revenue, capex per MW, capex per disclosed GPU, etc.
Eval agent model family	Must differ from extractor. Choice deferred until extractor backend is chosen.
Block vs flag on eval disagreement	Trade-off: blocking is safer but creates a stuck-row tail; flagging is permissive but risks releasing bad data.
Git LFS for source PDFs	Recommended yes. Final call deferred.
Live formula evaluation strategy	LibreOffice headless recalc (recommended) vs reimplementing rules in Python. Final call deferred.

10. What I Want to Implement

This section captures Kai's intent in his own framing, distilled from the design conversation:

- A collaborative GitHub project, built with friends, that automatically tracks AI-related capex spending across major hyperscalers and cloud / neocloud providers from their annual and quarterly reports.
- Claude is the primary development agent. Other model providers (e.g. Gemini for long-context reading, MiniMax for spreadsheet population) can be plugged in interchangeably to lower overall cost.
- Continuous operation: a background scheduler triggers extraction whenever a relevant company releases new quarterly disclosures. The scheduler runs on a server (free GitHub Actions tier) so live tokens are only spent when there is real work to do.
- A universal interchange protocol governs every model handoff. Each model produces proof-of-work artifacts (quotes, locators, source hashes) that the next stage can verify mechanically.
- A final eval / test agent re-checks proof-of-work against source materials before any result is released to the user.
- Validation and consistency-checking live in a hand-designed Excel template. openpyxl populates designated input cells; Excel formulas compute the check flags; the agent reads those flags. Excel is the single source of truth for validation rules.
- Both code and the workbook live in the same GitHub repository, with Git LFS for source PDFs and a CSV mirror for diff-friendly history. Workbook edits are explicitly serialized.
- Claude for Excel remains as an optional human interface for ad-hoc analyst questions against the populated workbook, but is not part of the automated pipeline.

- Next planning topic (after this memo): the collaboration protocol — how friends contribute, branch, review, label golden data, and own pieces of the system.

11. Immediate Next Steps

- Review and sign off on this memo (or annotate disagreements).
- Define the collaboration protocol (branching model, review rules, ownership of layers, golden-data labeling workflow).
- Lock the v1 target company list.
- Draft the source_data column schema as the formal protocol contract.
- Schedule the dedicated Excel template design session.
- Schedule the dedicated golden dataset labeling session.

End of memo. Version 0.5. To be revised after collaboration-protocol session.